FedUPI

Federated fraud shield

Financial Fraud Detection System using Federated Learning on UPI

User guide: setup, pages, AI models and how it works

Covers	Quick start - Terminal commands - Every page explained - AI model details - How a fraud decision is made - Datasets - Results - Limitations
Stack	FastAPI + PyTorch + scikit-learn + SQLite (backend), React + Vite + Recharts (frontend)
Scope	P2P, merchant and investment UPI payments
Data	Synthetic UPI-style data (five simulated banks) and the public PaySim dataset

1. What the project does

FedUPI detects suspicious UPI payments. Five banks each hold their own transactions. Instead of pooling that private data, every bank trains a copy of the same AI model locally and sends only the model weights to a server. The server averages the weights into one global model (**federated learning**) and sends it back. The result is a fraud model that has learned from all banks while no transaction ever left its bank.

The trained model is wrapped in a web application: analysts can check a single payment, upload files of payments, watch a live feed, review alerts, download fraud reports and even retrain the federated model from the browser.

Main ideas

- **Federated learning** (FedAvg, FedProx) - banks share weights, never data.
- **Secure aggregation** - the server only sees the sum of all banks' updates, not any single bank's update.
- **Two AI models** - a supervised fraud classifier and an unsupervised autoencoder that spots unusual behaviour.
- **Business risk policy** - amount tiers and repeated-payment rules on top of the model, with a risk percentage on every payment.
- **Explainability** - every flagged payment lists the reasons in plain words.

2. Quick start - the terminal commands

You need Python 3.11 or newer and Node.js 20 or newer installed. Open **two** PowerShell terminals in the project folder (`S:\prasanth\fedupi\fedupi`) and run one command in each. The first run installs everything and trains the model; after that they simply start.

Terminal	Command	What it does
1 - Backend	.\backend.ps1	Creates the Python environment, installs packages, trains the model if missing, starts the API on http://127.0.0.1:8000 (docs at /docs).
2 - Frontend	.\frontend.ps1	Installs npm packages on first run and starts the web app on http://127.0.0.1:5173.

Then open **http://127.0.0.1:5173** in your browser.

If PowerShell blocks the scripts, run this once and try again: Set-ExecutionPolicy -Scope CurrentUser RemoteSigned

First use, step by step

- Open the app and choose **Create an account**. Enter your name, email and a password of at least 8 characters.
- Open the menu with the button at the top left. Pick a page; the menu closes and the page opens.
- Go to **Live Monitor** and press **Simulate traffic** to see payments being scored.
- Go to **Upload & Datasets** and drop `samples/policy_rules_demo.csv` to see every risk rule fire.
- Open **Downloads** and export the flagged payments as Excel, CSV, Word, PDF or JSON.

Other useful commands

Purpose	Command (run inside the Backend folder)
Retrain the main model (synthetic UPI data)	.\.venv\Scripts\python.exe scripts\train.py
Train on real PaySim data without replacing the live model	.\.venv\Scripts\python.exe scripts\train.py --dataset paysim --no-activate
Switch the live model to PaySim / back to UPI	.\.venv\Scripts\python.exe scripts\train.py --use paysim (or --use upi)
Regenerate the sample datasets	.\.venv\Scripts\python.exe scripts\make_datasets.py
Run the automated tests (16 tests)	.\.venv\Scripts\python.exe -m pytest -q

3. The pages, explained

The menu button (top left) slides a menu in from the left. Choosing a page opens it and hides the menu again. The top bar also has a light/dark theme toggle and your profile menu.

Page	What you do there
Sign in / Create account	Register or sign in. Passwords are stored as scrypt hashes; sessions use signed tokens that last 12 hours. Every API route except the health check needs a signed-in user.
Dashboard	Overview: payments scored, high-risk and moderate-risk counts, open alerts, the risk-score stream, the risk-tier mix, the production model's test metrics and the five banks' sizes and fraud rates.
Live Monitor	Press **Simulate traffic** (or **Burst of 20**) to feed realistic payments through the model. Click any row to see why it was flagged. For simulated data it also shows how many real frauds were caught and how many false alarms occurred.
Check Transaction	Score one payment. Pick a preset (normal, repeat payments, Rs 85,000, Rs 1,20,000, collect scam, SIM-swap, mule burst) or fill the form. Give a Sender ID and Receiver ID to let the system read that user's earlier payments from the database. You get the risk %, the tier, the model score, the anomaly score and the reasons.
Upload & Datasets	Drop an Excel, CSV, JSON, TSV or Word-table file (up to 10 MB / 20,000 rows). It is stored in your library and scored immediately. From the library you can re-score, download the original, delete, or download only the flagged payments in any format.
Alert Queue	Flagged payments waiting for review. An analyst marks each one **Fraud** or **Dismiss**. Those labels are the feedback for future retraining. Has a download menu.
Downloads	Choose what to export (all flagged, high risk only, moderate only, low risk), the review status and the source file, preview the rows, then use the **Download report** dropdown: Excel, CSV, Word, PDF or JSON.
Federated Lab	Train the federated model from the browser. Pick FedAvg or FedProx, rounds, local epochs, learning rate, secure aggregation and optional differential privacy (clip, noise). A live chart shows convergence and the privacy budget epsilon. You can tick **promote** to replace the production model.
Model Insights	Test-set results: federated vs centralized vs local-only comparison, precision-recall curve, score distribution, what drives the fraud score, recall by fraud type, per-bank results, risk-policy results and privacy accounting.
Research & Papers	Architecture diagram, the six system modules, base papers and honest limitations.
Profile	Edit name, organization and role, change your password, choose light or dark theme, sign out.

4. How fraud is detected - step by step

A. Building the model (offline, federated)

- **Data per bank.** Five simulated banks each have their own payments, fraud rate and fraud mix.
- **Features.** Each payment becomes 22 numbers: amount (log and ratio to the user's average), hour, account age, payments in the last hour and 24 hours, time since the last payment, failed logins, UPI PIN failures, new device, new beneficiary, collect request, weekend, amount tier flags, repeated payments to the same receiver, and the payment category. Scales are fixed in code so no bank's statistics are shared.
- **Local training.** In every round each bank trains the model on its own data.
- **Aggregation.** Banks send weights only. With secure aggregation each bank masks its update with random numbers that cancel in the sum, so the server learns only the total. The server averages (FedAvg or FedProx) and sends the global model back.
- **Repeat** for 15 rounds, then choose the score threshold that gives the best F1 on validation data and measure on a held-out test set.

B. Scoring one payment (online)

- The payment arrives from the form, an uploaded file or the live monitor.
- Repeat-payment history is added: earlier payments by the same user in the last hour and 24 hours, and earlier payments by the same user to the same receiver in 24 hours (from the file's sender, receiver and timestamp columns, or from the database).
- The classifier outputs a fraud score (0-100%); the autoencoder outputs an anomaly score. They are fused with a weight chosen on validation data (it can be 0).
- The risk policy turns the score and the rules into a tier and a risk percentage (next page).
- For flagged payments an attribution method lists the features that pushed the score up, written as plain-language reasons.
- The result is saved; flagged payments appear in the Alert Queue.

5. Risk tiers, risk % and repeat rules

Tier	Rule	Action	Risk %
High risk	Amount above Rs 99,999 (Rs 1,00,000 and up). This is the only way to become High.	Block / hold	80 - 99
Moderate risk	Amount Rs 70,000 - 99,999, OR the AI model flags the payment, OR repeated payments (below).	Step-up authentication (OTP)	40 - 79
Low risk	Everything else. A payment below Rs 70,000 is never High because of its amount.	Allow	0 - 39

Repeated-payment rules

- **Same receiver:** two or more earlier payments by the same user to the same receiver within 24 hours makes the payment at least Moderate. A user paying a friend twice in a day stays Low; a third payment of similar size to the same unknown receiver is flagged.
- **Burst:** four or more earlier payments by the same user within one hour makes it at least Moderate (typical of mule accounts).
- The same repeat counts are also inputs to the AI model, so it learns which repetition patterns are really fraud.

The risk percentage

The percentage always agrees with the tier. The AI model's fraud score decides the position inside the tier's band. It is a risk index, not a calibrated probability of fraud.

Because High is reserved for amounts above Rs 99,999, a scam on a smaller amount (for example a Rs 40,000 SIM-swap) can reach Moderate at most. The model's own opinion is shown beside the risk % on the Check Transaction page.

6. AI model details

Model	Type	Role and details
Federated MLP classifier	Neural network, supervised	22 inputs, two hidden layers (64 and 32 units, ReLU, dropout 0.1), one output. Class-weighted binary cross-entropy to cope with rare fraud. Trained with SGD + momentum; 15 rounds, 2 local epochs per round.
Federated autoencoder	Neural network, unsupervised	22 inputs, bottleneck of 6. Trained on legitimate payments only; a high reconstruction error means unusual behaviour and gives an anomaly score. Fused into the final score only if it improves validation F1.
FedAvg	Aggregation	Weighted average of bank updates by sample count (McMahan et al., 2017).
FedProx	Aggregation	Adds a proximal term to keep local models close to the global model when banks differ (Li et al., 2020).
Secure aggregation	Privacy	Pairwise additive masking in fixed-point arithmetic; the server only sees the sum (Bonawitz et al., 2017). Simulated key agreement, no dropout recovery.
Differential privacy (option)	Privacy	Each update is clipped, Gaussian noise is added, and a Renyi-DP accountant reports client-level (epsilon, delta).
Gradient x input attribution	Explainability	Shows which input features raised a payment's fraud score; turned into reasons.
Gradient boosting	Baseline	Centralized reference model that sees all pooled data (not federated).

Measured results (held-out synthetic test set, 7915 payments)

Approach	Precision	Recall	F1	PR-AUC
Local-only (each bank alone, mean)	75.2%	68.5%	71.7%	0.770
Federated FEDAVG	80.5%	70.0%	74.9%	0.820
Federated FedProx	80.8%	71.4%	75.8%	0.817
Federated FedAvg + Secure Aggregation	80.5%	70.0%	74.9%	0.820
Federated FedAvg + DP (clip 1.0, z 0.3) (eps 143.36)	69.3%	71.0%	70.2%	0.746
Federated FedAvg + DP + SecAgg (clip 1.0, z 1.0) (eps 24.83)	43.1%	53.4%	47.7%	0.385
Centralized MLP (all data pooled)	77.1%	86.9%	81.7%	0.911
Centralized Gradient Boosting	85.6%	85.9%	85.7%	0.920

- Federated training beats banks working alone and approaches centralized training, without pooling any data.
- Differential privacy costs accuracy. With only five banks, epsilon 143 is no real guarantee and epsilon about 25 already hurts the model a lot.
- With the risk policy on the test set: High 107 payments (24 real fraud), Moderate 276 (182 fraud), Low 7532 (77 fraud). Flagged (Moderate + High) precision 53.8%, recall 72.8%.

- Only about 22% of High-tier payments are real fraud: 83 legitimate large payments are High because of the amount rule alone.
- Weak spots: stealthy fraud, QR phishing, and the first payment of a repeat burst (nothing has repeated yet).

7. Datasets and file format

Ready-to-upload files are in the `samples` folder:

File	Content
upi_test_transactions.xlsx / .csv / .json	About 2,100 mixed payments from five banks with a true fraud label (unseen by the model).
policy_rules_demo.csv / .xlsx	Hand-made rows: every amount tier, six repeat payments to one receiver, a seven-payment burst, scams and normal traffic.
upi_training_dataset.csv	The roughly 52,000-row dataset the model trains on.

Required columns for uploads

```
amount_inr, amount_to_average_ratio, transaction_hour, account_age_days,
transactions_last_hour, minutes_since_last_transaction,
failed_login_attempts_24h, upi_pin_failures_24h, is_new_device,
is_new_beneficiary, is_collect_request, is_weekend, merchant_category
```

merchant_category must be `p2p`, `merchant` or `investment`; other categories are rejected. **Recommended:** `sender_id`, `receiver_id`, `timestamp` - with these the system calculates the repeat-payment counts itself. **Optional:** `transaction_id`, `bank`, `is_fraud` (true label, used only for comparison).

Public datasets

- PaySim (real mobile-money simulator data): https://www.kaggle.com/datasets/ealaxi/paysim1 - supported through `--dataset paysim`.
- Credit card fraud (ULB): https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud - needs its own loader.
- IEEE-CIS fraud detection: https://www.kaggle.com/c/ieee-fraud-detection - needs its own loader.

No public dataset with real labelled UPI fraud exists, which is why this project uses synthetic UPI-style data. Kaggle requires a free login to download.

Download formats for reports

Excel (.xlsx), CSV, Word (.docx), PDF and JSON. Each report lists reference, time, bank, category, amount, risk %, tier, decision, review status and the reasons.

8. Project structure

```
fedupi/
  backend.ps1, frontend.ps1     one command per terminal
  Backend/
    app/api/                    REST endpoints (auth, scoring, datasets, exports, federated)
    app/data/                   generator, features, repeat-payment behaviour, PaySim loader
    app/ml/                     networks, federated engine, secure aggregation,
                                privacy accountant, risk policy, experiments
    app/services/               scoring, database, auth, uploads, exports, training jobs
    scripts/                    train.py, make_datasets.py
    tests/                      16 automated tests
    artifacts/                  trained model and report
  Frontend/src/pages/           Dashboard, Live Monitor, Check, Upload, Alerts,
                                Downloads, Federated Lab, Insights, Research, Profile
  samples/                      ready-to-upload datasets
```

Tech stack

Layer	Tools
AI / ML	PyTorch, scikit-learn, NumPy, pandas
Federated + privacy	Own implementation: FedAvg, FedProx, secure aggregation, DP accountant
Backend	FastAPI, Pydantic, SQLAlchemy + SQLite
Reports	openpyxl (Excel), python-docx (Word), ReportLab (PDF)
Frontend	React 18, Vite, React Router, Recharts

9. Troubleshooting

Problem	Fix
Scripts will not run in PowerShell	Set-ExecutionPolicy -Scope CurrentUser RemoteSigned, then run the command again.
Page says API offline	Start the backend terminal first (.\backend.ps1) and wait until it prints the address.
Port 8000 or 5173 is busy	Close the earlier terminal running the same command, then start again.
Upload rejected: missing columns	Add the required columns listed in section 7 or start from a sample file.
Upload rejected: unsupported merchant_category	Use only p2p, merchant or investment.
Signed out unexpectedly	Sessions last 12 hours; sign in again.

10. Limitations

- The UPI data is synthetic; results show the method, not real-world accuracy.
- All banks are simulated in one process on one computer.
- Secure aggregation uses simulated key agreement and has no dropout recovery. Model updates can still leak information; differential privacy has a large accuracy cost with five banks.
- The risk % is a risk index, not a calibrated probability. The amount rule makes many legitimate large payments High risk by design.
- The scoring API sees payment details in clear; this is a bank-side deployment, not private inference.

Base papers: McMahan et al. 2017 (FedAvg); Li et al. 2020 (FedProx); McMahan et al. 2018 (DP-FedAvg); Bonawitz et al. 2017 (secure aggregation); Yang et al. 2019 (federated learning concepts; FFD credit-card fraud); Lopez-Rojas et al. 2016 (PaySim). Verify citations before using them in a report.